

A Mini Project Report on

QUIZ GAME

Interactive Multiple Choice Quiz in Python

Submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology

Branch: Computer Science and Engineering (Artificial Intelligence and Machine Learning)

Submitted by

Aditya Kumar Singh

Register No: 26BAI10613

Institution: VIT Bhopal University

Academic Year: 2026–2027

VITyarthi • Build Your Own Project

CERTIFICATE

This is to certify that the project report titled "**Quiz Game – Interactive Multiple Choice Quiz in Python**" is the bonafide work of **Aditya Kumar Singh** (Register No: **26BAI10613**), a student of Computer Science and Engineering (Artificial Intelligence and Machine Learning) at VIT Bhopal University, submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology during the academic year 2026–2027.

Guide

Head of Department

Submitted for the University Examination held on: _____

Internal Examiner

External Examiner

DECLARATION

I hereby declare that the project report titled "**Quiz Game – Interactive Multiple Choice Quiz in Python**" submitted to VIT Bhopal University for the award of the degree of Bachelor of Technology is a record of original work carried out by me. I further declare that this project has not been submitted, either in part or in whole, for the award of any other degree or diploma.

Place: Bhopal

Date: 30/09/2026

Aditya Kumar Singh

26BAI10613

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty members and the VIT Bhopal University academic team for providing the opportunity and guidance required to complete this project. I am thankful for the learning environment that helped me apply Python programming concepts to a practical problem.

I also thank my classmates, friends, and family for their support and encouragement during the development and testing of the application.

Aditya Kumar Singh

ABSTRACT

This project presents the design and implementation of a console-based Quiz Game developed using Python. The application provides an interactive way for a user to answer multiple-choice questions, receive immediate feedback, and view a final score, percentage, and grade. A question-bank feature also allows new questions to be added and all stored questions to be displayed. Questions are persisted in a JSON file so that changes to the question bank remain available between program executions.

The implementation uses core Python concepts including classes, functions, loops, conditional statements, lists, dictionaries, random sampling, exception handling, input validation, and JSON file handling. The system is divided into separate modules for the user interface, quiz logic, question-bank operations, result calculation, storage, and validation. This modular structure satisfies the project requirement for a clean and maintainable implementation while keeping the program understandable for a first-year programming course.

The resulting application is lightweight, requires no external runtime packages, and can be executed on a standard computer with Python 3.8 or later. Future improvements can include a GUI, subject-wise categories, timers, a leaderboard, and a web interface.

Keywords: Python, Quiz, Multiple Choice, JSON, OOP, Input Validation, Console Application

TABLE OF CONTENTS

Section	Page
1 Introduction	1
2 Requirements	3
3 System Design	5
4 Implementation	8
5 Results and Testing	11
6 Conclusion and Future Scope	13
References	14

CHAPTER 1: INTRODUCTION

1.1 Overview

Quiz Game is a Python-based console application designed to provide a simple and interactive multiple-choice quiz experience. The program loads questions from a JSON question bank, selects a requested number of questions at random, accepts answers from the user, gives immediate feedback, and calculates a final score.

The project applies fundamental programming concepts to a complete small-scale application. The design separates quiz logic, input validation, user interface, result calculation, and data storage into independent modules. This makes the project easier to understand, test, maintain, and extend.

1.2 Problem Statement

Students often need a quick way to test their knowledge before examinations. Manual quizzes require preparation and checking, while a basic digital quiz can provide immediate feedback and a score. The project solves this problem by providing a lightweight application for attempting questions, checking answers, and reviewing performance.

1.3 Objectives

- Design and develop a Python-based interactive quiz application.
- Apply classes, functions, loops, conditional statements, lists, dictionaries, and file handling.
- Provide a clear console input/output workflow.
- Validate user inputs and handle invalid entries safely.
- Provide question-bank operations and persistent JSON storage.
- Demonstrate modular software design and basic testing.

1.4 Scope

The application is intended for a single user at a time and runs through the console. It supports quiz attempts, question addition, question viewing, score calculation, grading, validation, and JSON persistence. It does not currently include user accounts, network access, or a graphical interface.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	10 GB free space
Display	1366 × 768 or better
Keyboard / Mouse	Standard keyboard and mouse

2.2 Software Requirements

Component	Specification
Operating System	Windows, macOS, or Linux
Language	Python 3.8 or higher
IDE / Editor	PyCharm, VS Code, IDLE, or any text editor
Runtime Libraries	Python standard library only
Testing	pytest optional

2.3 Functional Requirements

- **FR1:** Load questions from persistent JSON storage.
- **FR2:** Start a quiz using a random selection of available questions.
- **FR3:** Display four options and accept a user answer.
- **FR4:** Validate answers and provide immediate feedback.
- **FR5:** Calculate score, percentage, and grade.
- **FR6:** Add new questions to the question bank.
- **FR7:** Display all stored questions.
- **FR8:** Save changes to the question bank.

2.4 Non-Functional Requirements

- **Usability:** The menu and prompts use simple, clear console messages.
- **Reliability:** Invalid answer formats are handled without terminating the quiz.
- **Maintainability:** Features are separated into focused modules and classes.
- **Performance:** The application uses lightweight in-memory lists and local JSON storage.
- **Resource Efficiency:** No external runtime libraries or network services are required.
- **Error Handling:** Invalid numbers, missing data, and malformed JSON are handled safely.

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

The architecture uses separate modules for user interaction, quiz processing, question management, validation, result calculation, and persistent storage. This separation keeps responsibilities clear and makes individual components easier to test.

System Architecture: Quiz Game

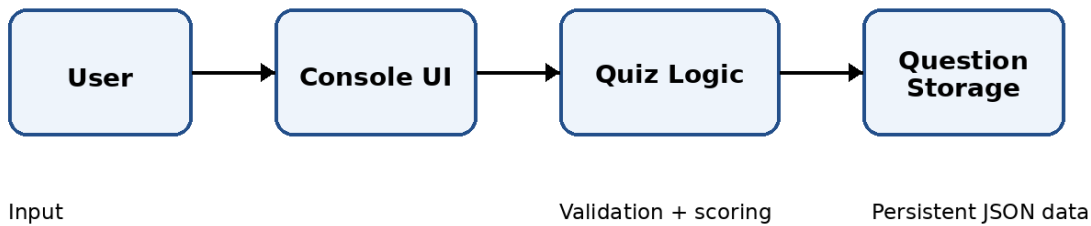


Fig. 3.1: System Architecture Diagram

3.2 Workflow

The workflow begins when the program loads the question bank. The user selects an operation from the main menu. During a quiz, questions are selected randomly, answers are validated, the score is updated, and the final result is displayed. When a new question is added, the updated question bank is written back to JSON.

Application Workflow

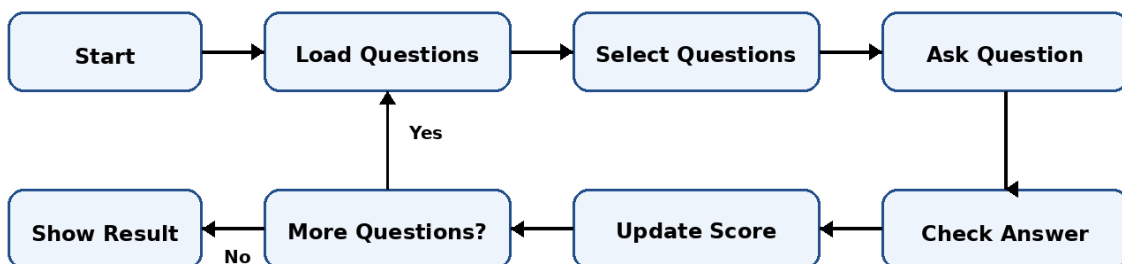


Fig. 3.2: Application Workflow

3.3 Module Description

Module	Responsibility
main.py	Controls the main menu and connects the modules.
question_bank.py	Loads and manages questions and addition of new questions.
quiz.py	Runs a quiz session and updates the score.
result.py	Calculates percentage and grade.

storage.py	Reads and writes the JSON question bank.
validator.py	Validates numeric and text input.
ui.py	Handles console presentation and user prompts.

CHAPTER 4: IMPLEMENTATION

4.1 Design Decisions and Rationale

- **JSON storage:** Chosen because it is simple, human-readable, and sufficient for a small first-year project.
- **Modular structure:** Separates responsibilities instead of placing the whole program in one large file.
- **Random selection:** Prevents every quiz attempt from following exactly the same question order.
- **Input validation:** Prevents invalid option values from corrupting the quiz flow.
- **Console interface:** Keeps the implementation focused on Python fundamentals without requiring GUI frameworks.

4.2 Key Implementation

The core project uses seven focused Python modules. The following excerpt shows the central quiz loop:

```
for index, question in enumerate(picked, 1):
    ui.show_question(index, question)

    while True:
        raw_answer = input(" Enter your answer (1-4): ")
        answer = InputValidator.option_number(raw_answer)
        if answer is not None:
            break
        print(" Invalid input. Please enter a number from 1 to 4.")

    if answer == question["answer"]:
        print(" Correct!")
        score += 1
    else:
        correct = question["options"][question["answer"] - 1]
        print(f" Wrong! Correct answer was: {correct}")
```

4.3 Result Calculation

```
percentage = (score / total * 100) if total else 0

if percentage >= 90:
    grade = "A+"
elif percentage >= 80:
    grade = "A"
elif percentage >= 60:
    grade = "B"
elif percentage >= 40:
    grade = "C"
else:
    grade = "F"
```

4.4 Data Storage

Questions are represented as dictionaries containing a question string, a list of four options, and an integer answer from 1 to 4. The collection of dictionaries is stored in data/questions.json.

CHAPTER 5: RESULTS AND TESTING

5.1 Sample Output

The application was executed locally using Python. A sample run was generated from the project code and is included below.

```
=====
                        QUIZ GAME
=====
1. Start Quiz
2. Add New Question
3. Show All Questions
4. Exit
Enter your choice: How many questions? (1-10):
=====
                        QUIZ STARTED
=====
Total questions: 3

Q1. Which is the largest ocean in the world?
1. Atlantic
2. Indian
3. Arctic
4. Pacific
Enter your answer (1-4): Wrong! Correct answer was: Pacific

Q2. Who is commonly credited with inventing the telephone?
1. Edison
2. Tesla
3. Bell
4. Newton
Enter your answer (1-4): Correct!

Q3. Pizza is traditionally associated with which country?
1. France
2. USA
3. Italy
4. Spain
Enter your answer (1-4): Wrong! Correct answer was: Italy

=====
                        RESULT
=====
Score      : 1 / 3
Percentage : 33.3%
Grade      : F

=====
                        QUIZ GAME
=====
1. Start Quiz
2. Add New Question
3. Show All Questions
4. Exit
Enter your choice: Bye! Thanks for playing.
```

Fig. 5.1: Sample console execution

5.2 Testing Approach

Testing focused on score calculation, grade calculation, valid option input, invalid option input, and positive-number validation. The project includes tests/test_quiz.py for repeatable automated checks.

Test Case	Input / Condition	Expected Result	Status
Correct option	1–4 matching answer	Score increases by 1	Pass
Wrong option	1–4 not matching answer	Wrong feedback shown	Pass
Invalid option	0, 5, or text	Input rejected and re-entered	Pass
Positive number	Valid quiz count	Quiz starts with requested count	Pass
Zero/negative count	0 or negative	Input rejected	Pass
Question addition	Valid question + 4 options	Question saved to JSON	Pass

Empty question	Blank question text	Question not added	Pass
----------------	---------------------	--------------------	------

5.3 Challenges Faced

- Keeping user input from terminating the application when non-numeric data is entered.
- Separating the original single-file idea into modules without making the project unnecessarily complex.
- Persisting newly added questions so that they remain available after restarting the program.
- Keeping the implementation suitable for a beginner while still meeting modular project requirements.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Quiz Game was successfully designed and implemented using Python. The application meets the main project objectives by providing an interactive quiz workflow, immediate feedback, score calculation, input validation, question-bank management, and persistent storage. The modular structure demonstrates how classes, functions, data structures, loops, conditions, and file handling can be combined into a complete application.

Testing of the core validation and result components confirms that the program handles both valid and invalid inputs in the expected manner. The project also provides a foundation that can be extended without rewriting the complete application.

6.2 Learnings and Key Takeaways

- Learned how to divide a Python application into reusable modules.
- Practiced object-oriented programming through small focused classes.
- Used JSON for persistent local data storage.
- Implemented input validation and error-handling strategies.
- Understood the relationship between requirements, design, implementation, and testing.

6.3 Future Enhancements

- Add a graphical user interface using Tkinter.
- Add subject-wise categories such as Python, C, Mathematics, and General Knowledge.
- Add a timer for each question.
- Store users and scores in SQLite and build a leaderboard.
- Add login and multiple-user support.
- Convert the application into a web application using Flask or Streamlit.

REFERENCES

- [1] Reema Thareja, *Python Programming Using Problem Solving Approach*, Oxford University Press, 2017.
- [2] Mark Lutz, *Learning Python*, 5th Edition, O'Reilly Media, 2013.
- [3] Zed A. Shaw, *Learn Python 3 the Hard Way*, Addison-Wesley, 2017.
- [4] Python Software Foundation, *Python 3 Documentation*.
- [5] VITyarthi, *Build Your Own Project: General Project Instructions & Submission Guidelines*, provided project guideline document.
- [6] Python Project Template, *Mini Project Report Template*, provided academic template.